

Tensor Networks for quantum circuit simulation and quantum error correction

Jin-Guo Liu and Zhong-Yi Ni

Advanced Materials Thrust, Function Hub, HKUST(GZ)

Contents

Tensor Networks	2
Representations of Multilinear Algebra	2
Einsum Notation and Computational Complexity	5
Contraction order optimization and slicing	11
Data Compression and Tensor Decomposition	15
Automatic Differentiation	21
Quantum Circuit Simulation	23
Quantum States and Quantum Gates	23
Example: Hadamard test	26
Example: Quantum teleportation	27
Quantum channel simulation	28
Kraus operators	29
Tensor network representation of channels	29
Quantum Error Correction	31
Stabilizers and Quantum Codes	31
Surface code	33
Decoding problem	33
Tensor network decoder	35
Bibliography	35

Tensor Networks

Representations of Multilinear Algebra

A *tensor network* is a mathematical framework that represents multilinear algebra operations as intuitive graphical structures, where tensors become nodes and shared indices become connecting edges. This diagrammatic approach transforms complex high-dimensional contractions into accessible visual networks that expose underlying computational structure.

The framework exhibits remarkable universality, emerging across diverse domains: *einsum* notation[1] in numerical computing, *factor graphs*[2] in probabilistic inference, *sum-product networks* in machine learning, and *junction trees*[3] in graphical models. Tensor networks have revolutionized quantum circuit simulation[4], quantum error correction[5], neural network compression[6], and strongly correlated quantum materials[7].

(Definition 1) (**Multilinear algebra.**): *Multilinear algebra* is the study of functions that are linear in each of their arguments separately. A function $f : V_1 \times V_2 \times \dots \times V_k \rightarrow W$ is called *multilinear* if it is linear with respect to each argument when all other arguments are held fixed. That is, for any $i \in \{1, 2, \dots, k\}$ and fixed values of all arguments except the i -th, the function

$$v_i \mapsto f(v_1, \dots, v_{i-1}, v_i, v_{i+1}, \dots, v_k) \quad (1)$$

is linear. The canonical example is the inner product $\langle x, y \rangle$, which is *bilinear*—linear in x when y is fixed, and linear in y when x is fixed.

To build intuition, we begin by recalling that a function f is *linear* if it satisfies the fundamental properties of additivity, where $f(x + y) = f(x) + f(y)$ for any vectors x and y , and homogeneity, where $f(\alpha x) = \alpha f(x)$ for any scalar α .

Consider the chain multiplication of matrices:

$$O_{in} = \sum_{j,k,l,m} A_{ij} B_{jk} C_{kl} D_{lm} E_{mn} \quad (2)$$

This expression exemplifies a *tensor contraction*—a multilinear map where the output O_{in} depends linearly on each input tensor A, B, C, D, E . The summation over repeated indices (j, k, l, m) constitutes the “contraction” operation that eliminates internal degrees of freedom.

Exercise: Multilinear algebra

Identify the multilinear algebra in the following expression

1. scalar product: $f(x, y, z) = xyz$
2. trace multiplication: $f(A, B, C) = \text{tr}(ABC)$
3. kronecker product: $f(X, Y, Z) = X \otimes Y \otimes Z$
4. addition: $f(x, y, z) = x + y + z$

Solution:

1. Yes
2. Yes
3. Yes

(Definition 2) (**Tensor Network**): A *tensor network* is a mathematical framework for defining multilinear maps, which can be represented by a triple $\mathcal{N} = (\Lambda, \mathcal{T}, V_0)$, where:

- Λ is the set of variables (indices) present in the network $\mathcal{N}$
- $\mathcal{T} = \{T_{V_k}\}_{k=1}^K$ is the set of input tensors, where each tensor T_{V_k} is associated with the index set V_k
- V_0 specifies the indices of the output tensor

Each tensor $T_{V_k} \in \mathcal{T}$ is labeled by a set of variables $V_k \subseteq \Lambda$, where the cardinality $|V_k|$ equals the rank of T_{V_k} . The variables in $\Lambda \setminus V_0$ are the internal indices that will be contracted (summed over).

(Definition 3) (**Tensor network contraction**): The multilinear map, or *contraction*, applied to a tensor network $\mathcal{N} = (\Lambda, \mathcal{T}, V_0)$ is defined as:

$$T_{V_0} = \text{contract}(\Lambda, \mathcal{T}, V_0) := \sum_{m \in \text{dom}(\Lambda \setminus V_0)} \prod_{T_V \in \mathcal{T}} T_V(m) \quad (3)$$

where m represents an assignment of values to the variables in $\Lambda \setminus V_0$, and the summation runs over all possible such assignments.

Example: Matrix multiplication can be described as the contraction of a tensor network:

$$(AB)_{\{i,k\}} = \text{contract}(\{i, j, k\}, \{A_{\{i,j\}}, B_{\{j,k\}}\}, \{i, k\}) \quad (4)$$

where matrices A and B are input tensors with variable sets $\{i, j\}$ and $\{j, k\}$ respectively, which are subsets of $\Lambda = \{i, j, k\}$. The output tensor has variables $\{i, k\}$ and the summation runs over variables $\Lambda \setminus \{i, k\} = \{j\}$. The contraction corresponds to:

$$(AB)_{\{i,k\}} = \sum_j A_{\{i,j\}} B_{\{j,k\}} \quad (5)$$

In the following discussion, we will ignore the set notation and use the more compact subscript notation for the variable sets: $(AB)_{ik} = \sum_j A_{ij} B_{jk}$.

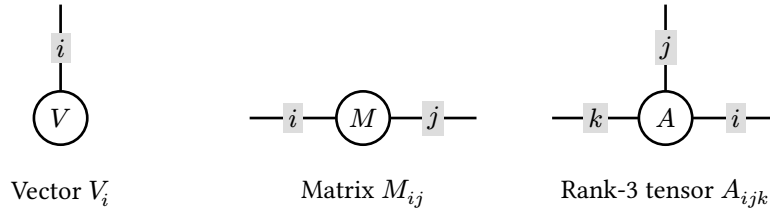
Generalization: This definition introduces a minor generalization of the standard tensor network definition commonly used in physics. It allows a label to appear more than twice across the tensors in the network, deviating from the conventional practice of restricting each label to two appearances. This generalized form, while maintaining the same level of representational power, has been demonstrated to potentially reduce the network's treewidth, a metric that measures its connectivity.

(Definition 4) (**Tensor network diagram.**): A *tensor network diagram* is a graphical representation of a tensor network that provides an intuitive visualization of the mathematical structure. In this representation:

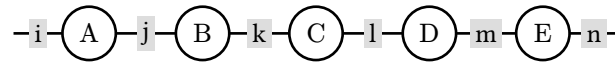
- Each tensor $T_i \in \mathcal{T}$ becomes a *node* (represented as a circle or geometric shape)
- Each index of tensor T_i becomes an *edge* or “leg” extending from the corresponding node
- Internal indices become *internal edges* connecting different nodes that share the same index
- External indices become *external edges* representing the dimensions of the final result

The diagram transforms abstract index manipulations into intuitive network topology where the computational structure becomes immediately apparent.

This graphical notation extends the algebraic concept of tensor contraction through powerful visual representation, rendering complex multilinear operations more accessible and revealing underlying computational structure that guides optimization strategies.



The diagrammatic representation of Equation 2 reveals the underlying structure more clearly:



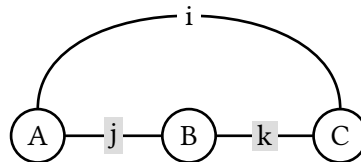
This visual approach proves indispensable when analyzing complex tensor contractions, as illustrated in the following example.

Example: Trace Permutation Rule

The cyclic property of matrix traces provides an excellent introduction to tensor network reasoning. Consider three square matrices A , B , and C of compatible dimensions. The trace permutation rule states:

$$\text{tr}(ABC) = \text{tr}(CAB) = \text{tr}(BCA) \quad (6)$$

In the algebraic proof, this identity follows from the definition of matrix multiplication and trace, but requires careful index manipulation. In the tensor network approach, the graphical approach reveals why this identity holds at a topological level.



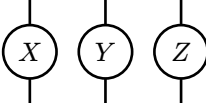
The tensor network reveals the essential structure—a closed loop where indices connect cyclically. The trace operation corresponds to this loop contraction, and the cyclic symmetry becomes immediately apparent.

Key lesson: Regardless of where we “cut” the loop to begin algebraic evaluation, the result remains identical. The three expressions $\text{tr}(ABC)$, $\text{tr}(CAB)$, and $\text{tr}(BCA)$ represent the same geometric object viewed from different starting points. This topological invariance provides deeper insight than purely algebraic derivations.

Exercise: Tensor network diagram

Draw the tensor network diagram for the multilinear algebra operations in the first exercise.

Solution:

1. 
2. see above.
3. 

Einsum Notation and Computational Complexity

(Definition 5) (**Einsum notation.**): *Einsum notation* (short for “Einstein summation notation”) is a compact string-based representation for specifying tensor network topologies and contractions. The notation consists of:

- Input tensor specifications on the left side, separated by commas
- The symbol $\rightarrow$ separating inputs from outputs
- Output tensor specification on the right side
- Each character represents a unique tensor index

The contraction rules are:

- Indices appearing only on the right side become output dimensions
- Indices appearing on the left but not on the right are summed over (contracted)

For example, matrix multiplication $C = AB$ is represented as $ij, jk \rightarrow ik$, where i and k are preserved output indices, and j is contracted (summed over).

In computational practice, this notation provides a concise way to encode contraction structure through simple syntax, making complex tensor operations easily readable and implementable.

Exercise: Einsum notation

1. Express the multilinear algebra operations in the first exercise using einsum notation.

Solution:

1. $.,, \rightarrow$
2. $ij, jk, ki \rightarrow$
3. $ij, kl, mn \rightarrow ikmjln$
2. Identify the operations:
 1. $aa \rightarrow$
 2. $aa \rightarrow a$
 3. $ab \rightarrow$

4. $ijl, jkl \rightarrow ikl$
5. $i \rightarrow ii$
6. $\rightarrow i$
7. $j, j \rightarrow$
8. $i, j \rightarrow ij$

Solution:

1. $aa \rightarrow$: trace of a matrix
2. $aa \rightarrow a$: diagonal extraction of a matrix
3. $ab \rightarrow$: summation of a matrix
4. $ijl, jkl \rightarrow ikl$: batched matrix multiplication
5. $i \rightarrow ii$: create a diagonal matrix from a vector
6. $\rightarrow i$: broadcast a scalar into a vector
7. $j, j \rightarrow$: dot product of two vectors
8. $i, j \rightarrow ij$: outer product of two vectors

The following examples use the [OMEinsum](#) package to demonstrate tensor network specification, contraction order optimization, and execution. Tensor network topologies can be defined using either the convenient ein string literal or the more flexible EinCode constructor for programmatic construction.

```
julia> using OMEinsum

julia> code = ein"ab, bc, cd -> ad" # using string literal
ab, bc, cd -> ad

julia> EinCode([[1,2], [2, 3], [3, 4]], [1, 4]) # alternatively
1*2, 2*3, 3*4 -> 1*4
```

Its defining properties can be obtained with the `getixsv` and `getiyv` functions.

```
julia> getixsv(code) # get the indices of the input tensors
3-element Vector{Vector{Char}}:
 ['a', 'b']
 ['b', 'c']
 ['c', 'd']

julia> getiyv(code) # get the indices of the output tensor
2-element Vector{Char}:
 'a': ASCII/Unicode U+0061 (category Ll: Letter, lowercase)
 'd': ASCII/Unicode U+0064 (category Ll: Letter, lowercase)
```

The complexity of the contraction can be computed with the `contraction_complexity` function. It requires the sizes of the indices, which can be specified with a dictionary that maps the indices to their sizes. Here, we use the `uniformsize` function to specify that all indices have the same size.

```
julia> label_sizes = uniformsize(code, 100) # define the sizes of the indices
Dict{Char, Int64} with 4 entries:
```

```

'a' => 100
'c' => 100
'd' => 100
'b' => 100

julia> contraction_complexity(code, label_sizes)
Time complexity: 2^26.575424759098897
Space complexity: 2^13.287712379549449
Read-write complexity: 2^15.287712379549449

```

Tensor contraction complexity can be analyzed through three complementary lenses:

Time complexity ($\approx 100^4$ operations): The total number of floating-point operations (FLOPs) required for contraction. For einsum operations, this generally equals the product of all unique index dimensions—each combination of index values requires one multiplication and addition. Crucially, intelligent contraction ordering can dramatically reduce this from exponential to polynomial scaling.

Space complexity ($\approx 100^2$ elements): The peak memory footprint needed to store the largest intermediate tensor during contraction. This represents the computational “bottleneck” that determines whether a calculation is feasible on available hardware.

Read-write complexity ($\approx 4 \times 100^2$ operations): The total memory bandwidth consumed by transferring all intermediate results between storage and processor. On modern architectures where memory access often dominates arithmetic costs, this metric frequently determines practical performance.

While EinCode objects are callable and can directly perform contractions:

```

julia> code(randn(2, 2), randn(2, 2), randn(2, 2)) # not recommended
2×2 Matrix{Float64}:
-0.974692  3.06151
-0.674225  1.40281

```

This direct approach is **strongly discouraged** because OMEinsum defaults to an unoptimized contraction order that can be exponentially inefficient. The naive left-to-right evaluation frequently creates unnecessarily large intermediate tensors.

A superior approach explicitly specifies the contraction order using parentheses to guide the computation:

```

julia> nested_code = ein"(ab,bc),cd->ad"
ac, cd -> ad
├─ ab, bc -> ac
│   ├─ ab
│   └─ bc
└─ cd

```

The resulting NestedEinsum object represents a structured two-step contraction: first computing the intermediate tensor from the first two inputs, then contracting this result with the third tensor. This explicit ordering achieves dramatic complexity reduction:

```
julia> contraction_complexity(nested_code, label_sizes)
Time complexity: 2^20.931568569324174
Space complexity: 2^13.287712379549449
Read-write complexity: 2^15.872674880270605
```

Beyond theoretical complexity improvements, practical performance gains are even more substantial. OMEinsum leverages optimized BLAS routines for binary tensor contractions, leading to remarkable speedups:

```
julia> using BenchmarkTools

julia> @btime code(randn(100, 100), randn(100, 100), randn(100, 100)); #
unoptimized
 86.418 ms (36 allocations: 385.48 KiB)

julia> @btime nested_code(randn(100, 100), randn(100, 100), randn(100, 100)); #
optimized
 133.167 μs (157 allocations: 486.06 KiB)
```

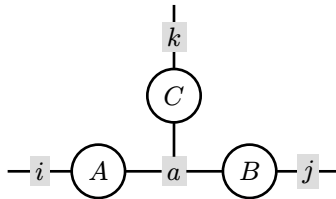
This represents over 600× speedup, demonstrating how proper contraction ordering transforms intractable computations into practical ones.

Example A: Star Contraction Pattern

Star contractions demonstrate how tensor networks can create higher-dimensional outputs from matrix inputs. Given three matrices $A, B, C \in \mathbb{R}^{n \times n}$, the star contraction produces a rank-3 tensor:

$$O_{ijk} = \sum_a A_{ia} B_{aj} C_{ak} \quad (7)$$

Geometric interpretation: Three matrices “radiate” from a central summation index a , creating a star-like topology. This pattern frequently appears in quantum many-body problems and machine learning applications.



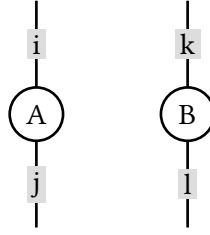
The einsum notation is $ai, aj, ak \rightarrow ijk$ with time complexity $O(n^4)$, where the shared index a creates the characteristic “star” topology.

Example B: Kronecker Product Structure

The Kronecker product illustrates tensor networks with disconnected components. For matrices $A, B \in \mathbb{R}^{n \times n}$, their Kronecker product creates:

$$C_{ijkl} = A_{ij} B_{kl} \quad (8)$$

Structural insight: The absence of shared indices reflects the direct product structure—no information flows between the matrix components. This disconnected topology contrasts sharply with the connected star pattern.



The einsum notation $ij,kl \rightarrow ijkl$ reveals the $O(n^4)$ complexity, which arises from forming all possible index combinations rather than performing contractions. This disconnected structure makes the Kronecker product trivially parallelizable.

Tensor Network Contraction is #P-Complete

Understanding the computational complexity of tensor network contraction requires connecting it to established hard problems in computer science. We achieve this through the concept of computational reduction.

Computational reduction: To establish computational hardness, we use the technique of *reduction*. If we can transform any instance of problem $\mathcal{A}$ into an instance of problem $\mathcal{B}$ using polynomial-time operations, then $\mathcal{B}$ is at least as hard as $\mathcal{A}$. When $\mathcal{A}$ is known to be computationally intractable, this proves that $\mathcal{B}$ is also intractable.

(Definition 6) (2-SAT formula.): A 2-SAT formula is a Boolean formula in conjunctive normal form (CNF) where each clause contains at most two literals. For those not familiar with Boolean logic:

- A *literal* is a Boolean variable or its negation ($\neg$)
- A *clause* is a disjunction (logical or: $\vee$) of literals
- A Boolean formula in *conjunctive normal form* is a conjunction (logical and: $\wedge$) of clauses

The *satisfiability problem* asks whether there exists an assignment of truth values that makes the formula true. The *counting problem* asks how many such satisfying assignments exist.

Example: Consider the 2-SAT formula:

$$(x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (x_3 \vee x_4) \wedge (x_4 \vee x_5) \wedge (x_5 \vee x_1) \wedge (x_3 \vee \neg x_5) \quad (9)$$

This formula contains six clauses and five Boolean variables. A satisfying assignment is: $x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0, x_5 = 1$. A non-satisfying assignment is: $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 0, x_5 = 1$, since it violates the clauses $x_2 \vee x_3$, $x_3 \vee x_4$ and $x_3 \vee \neg x_5$.

Computational complexity: While determining satisfiability (finding any solution) for 2-SAT formulas is polynomial-time solvable, counting the number of satisfying assignments is #P-complete—a complexity class considered even more challenging than NP-complete problems.

(*Theorem 1*) (**#P-Completeness of Tensor Network Contraction.**): General tensor network contraction is #P-complete. That is, the problem of computing the scalar result of contracting a tensor network belongs to the #P-complete complexity class.

Proof: We establish #P-completeness through a polynomial-time reduction from the canonical #P-complete problem of counting satisfying assignments of 2-SAT Boolean formulas.

Reduction construction: Given a 2-SAT formula $c_1 \wedge c_2 \wedge \dots \wedge c_n$, we construct a tensor network that computes the number of its satisfying assignments.

Step 1: Variable encoding. Boolean variables $x_1, x_2, \dots, x_n$ map directly to tensor indices in the network, denoted as $\Lambda = \{x_1, x_2, \dots, x_n\}$.

Step 2: Clause encoding. Each clause becomes a rank-2 tensor encoding its truth table. For a clause c_i involving variables x_i, x_j , we construct tensor $T(c_i)$ indexed by x_i, x_j where entry $T_{v_i v_j}$ equals 1 if the clause is satisfied by assignment $x_i = v_i, x_j = v_j$, and 0 otherwise.

Step 3: Network contraction. The counting problem reduces to the tensor contraction:

$$\text{count} = \text{contract}(\{x_1, x_2, \dots, x_n\}, \{T(c_i) \mid i = 1 \dots n\}, \emptyset) \quad (10)$$

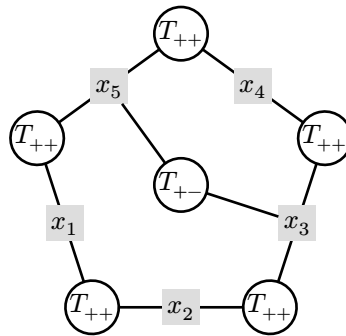
This contraction precisely counts satisfying assignments because each valid assignment contributes 1 to the sum (all clause tensors evaluate to 1), while invalid assignments contribute 0 (at least one clause tensor evaluates to 0).

Since counting satisfying assignments for 2-SAT is #P-complete, and this reduction can be performed in polynomial time, general tensor network contraction is also #P-complete. ■

To illustrate this reduction, we apply it to the formula in Equation 9. For the clause $(x_3 \vee \neg x_5)$, we construct tensor T_{+-} where entry (i, j) indicates whether the clause is satisfied when $x_3 = i$ and $x_5 = j$:

$$T_{+-} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \quad (11)$$

The complete tensor network for this formula is:

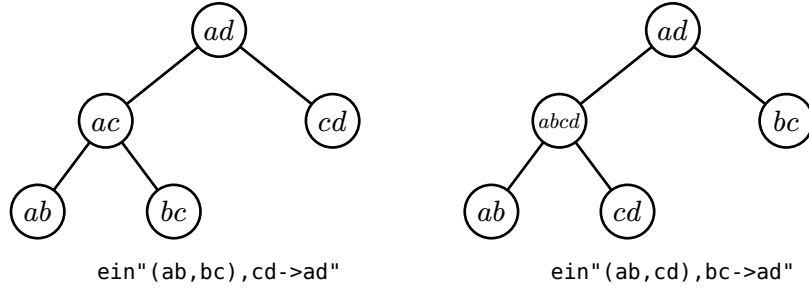


This #P-completeness result explains why approximate methods and heuristic algorithms dominate practical tensor network computations, motivating the optimization techniques discussed in subsequent sections.

Contraction order optimization and slicing

The computational cost of tensor network contraction depends critically on the chosen **contraction order**—the sequence in which pairwise tensor multiplications are performed. This order can be represented as a binary tree where leaves correspond to input tensors and internal nodes represent intermediate results.

Consider the contraction $\text{ein}''(ab, bc, cd \rightarrow ad)$, which admits multiple valid orderings with dramatically different costs:



The left ordering proves dramatically superior: it achieves $O(n^3)$ time and $O(n^2)$ space complexity by first contracting compatible matrices. The right ordering creates a $O(n^4)$ intermediate tensor through an inefficient Kronecker product, illustrating how ordering choice can determine computational feasibility.

Finding the globally optimal contraction order constitutes an NP-complete optimization problem[4]. Fortunately, near-optimal solutions often suffice for practical applications and can be obtained efficiently through sophisticated heuristic methods. Modern optimization algorithms have achieved remarkable scalability, successfully handling tensor networks with over 10^4 tensors[8],[9].

The optimal contraction order has a deep mathematical connection to the *tree decomposition*[4] of the tensor network's line graph.

(Definition 7) (Tree decomposition and treewidth.): A *tree decomposition* of a (hyper)graph $G = (V, E)$ is a tree $T = (B, F)$ where each node $B_i \in B$ contains a subset of vertices in V (called a “bag”), satisfying:

1. Every vertex $v \in V$ appears in at least one bag.
2. For each (hyper)edge $e \in E$, there exists a bag containing all vertices in e .
3. For each vertex $v \in V$, the bags containing v form a connected subtree of T .

The *width* of a tree decomposition is the size of its largest bag minus one. The *treewidth* of a graph is the minimum width among all possible tree decompositions.

The line graph of a tensor network is a graph where vertices represent indices and edges represent tensors sharing those indices. The relationship between a tensor network's contraction order and the tree decomposition of its line graph can be understood through several key correspondences:

- Each leg (index) in the tensor network becomes a vertex in the line graph, while each tensor becomes a hyperedge connecting multiple vertices.
- The tree decomposition's first two requirements ensure that all tensors are accounted for in the contraction sequence - each tensor must appear in at least one bag, with each bag representing a contraction step.

- The third requirement of the tree decomposition maps to an important constraint in tensor contraction: an index can only be eliminated after considering all tensors connected to it.
- For tensor networks with varying index dimensions, we can extend this relationship to weighted tree decompositions, where vertex weights correspond to the logarithm of the index dimensions.

The figure below illustrates these concepts with (a) a tensor network containing four tensors T_1 , T_2 , T_3 and T_4 and eight indices labeled A through H , (b) its corresponding line graph, and (c) a tree decomposition of that line graph.

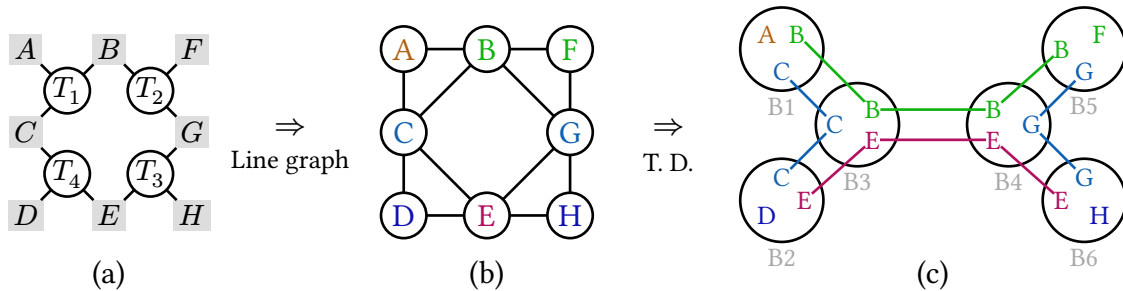
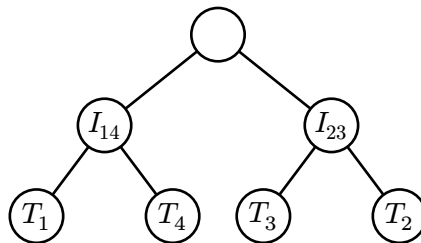


Figure 2: (a) A tensor network. (b) A line graph for the tensor network. Labels are connected if and only if they appear in the same tensor. (c) A tree decomposition (T. D.) of the line graph.

The tree decomposition in (c) consists of 6 bags, each containing at most 3 indices, indicating that the treewidth of the tensor network is 2. The tensors T_1 , T_2 , T_3 and T_4 are contained in bags B_1 , B_5 , B_6 and B_2 respectively. Following the tree structure, we perform the contraction from the leaves. First, we contract bags B_1 and B_2 into B_3 , yielding an intermediate tensor $I_{14} = T_1 * T_4$ (where “ $*$ ” denotes tensor contraction) with indices B and E . Next, we contract bags B_5 and B_6 into B_4 , producing another intermediate tensor $I_{23} = T_2 * T_3$ also with indices B and E . Finally, contracting B_3 and B_4 yields the desired scalar result.



Finding the optimal contraction order is almost equivalent to finding the minimal-width tree decomposition of the line graph. The log time complexity for the bottleneck contraction corresponds to the largest bag size in the tree decomposition. The log space complexity is equivalent to the largest separator (the set of vertices connecting two bags) size in the tree decomposition.

Heuristic methods for finding the optimal contraction order

OMEinsum provides multiple heuristic methods for finding the optimal contraction order. They are implemented in the dependency `OMEinsumContractionOrders`. To demonstrate the usage, we first generate a large enough random tensor network with the help of the `Graphs` package.

```
julia> using OMEinsum, Graphs

julia> function demo_network(n::Int)
    g = random_regular_graph(n, 3)
    code = EinCode([[e.src, e.dst] for e in edges(g)], Int[])
    sizes = uniformsize(code, 2)
end
```

```

        tensors = [randn([sizes[leg] for leg in ix]...) for ix in
getixsv(code)]
        return code, tensors, sizes
    end
demo_network (generic function with 1 method)

julia> code, tensors, sizes = demo_network(100);

julia> contraction_complexity(code, sizes)
Time complexity: 2100.0
Space complexity: 20.0
Read-write complexity: 29.231221180711184

```

We first generate a random 3-regular graph with 100 vertices. Then we associate each vertex with a binary variable and each edge with a tensor of size 2×2 . The time complexity without contraction order optimization is 2^{100} , which is equivalent to brute-force. The order can be optimized with the `optimize_code` function.

```

julia> optcode = optimize_code(code, sizes, TreeSA());

julia> cc = contraction_complexity(optcode, sizes)
Time complexity: 217.241796993093228
Space complexity: 213.0
Read-write complexity: 216.360864226366807

```

The `optimize_code` function takes three inputs: the `EinCode` object, the tensor sizes, and the contraction order solver. It returns a `NestedEinsum` object of time complexity $\sim 2^{17.2}$. It is much smaller than the number of vertices. It is a very reasonable number because the treewidth of a 3-regular graph is approximately upper bounded by $1/6$ of the number of vertices[10].

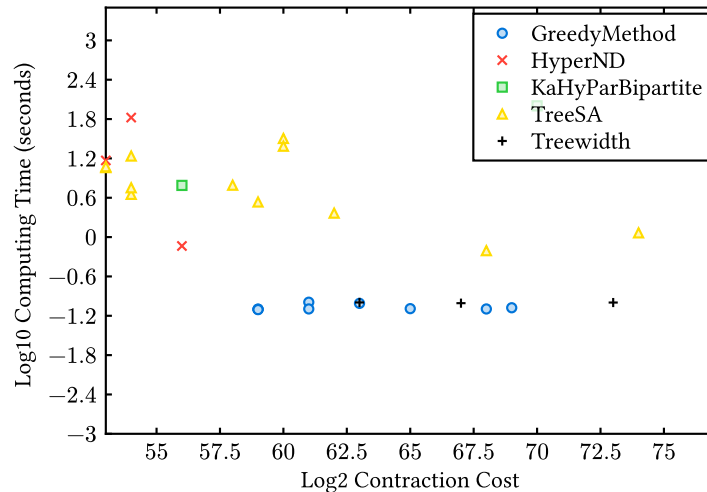


Figure 3: The contraction order quality measured by the space complexity (x -axis) and time complexity (y -axis) for different methods with different hyper-parameters. For details, please check GitHub repository [OMEinsumContractionOrdersBenchmark](#).

Among the available solver backends, TreeSA and HyperND usually provide the best contraction order quality. However, they are slow. For overhead sensitive applications, one can use GreedyMethod or Treewidth method.

In the following, we introduce the local search method TreeSA in detail.

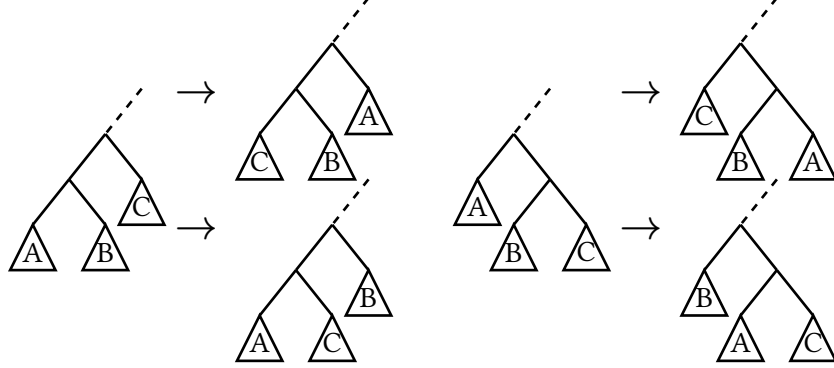


Figure 4: The four basic local transformations on the contraction tree, which preserve the result of the contraction.

The local search method[11] is a heuristic method based on the idea of simulated annealing. The method starts from a random contraction order and then applies the following four possible transforms as shown in Figure 4, which correspond to the different ways to contract three sub-networks:

$$\begin{aligned} (A * B) * C &= (A * C) * B = (C * B) * A, \\ A * (B * C) &= B * (A * C) = C * (B * A), \end{aligned} \quad (12)$$

where A, B, C are the sub-networks to be contracted. Due to the commutative property of the tensor contraction, such transformations do not change the result of the contraction. Even through these transformations are simple, all possible contraction orders can be reached from any initial contraction order. The local search method starts from a random contraction tree. In each step, the above rules are randomly applied to transform the tree and then the cost of the new tree is evaluated, which is defined as

$$\mathcal{L} = \text{tc} + w_s \text{sc} + w_{\text{rw}} \text{rwc}, \quad (13)$$

where w_s and w_{rw} are the weights of the space complexity and read-write complexity compared to the time complexity, respectively. Then the transformation is accepted with a probability given by the Metropolis criterion, which is

$$p_{\text{accept}} = \min(1, e^{-\beta \Delta \mathcal{L}}), \quad (14)$$

where β is the inverse temperature, and $\Delta \mathcal{L}$ is the difference of the cost of the new and old contraction trees. During the process, the temperature is gradually decreased, and the process stop when the temperature is low enough. Additionally, the TreeSA method supports the slicing technique. When the space complexity is too large, one can loop over a subset of indices, and then contract the intermediate results in the end. Such technique can reduce the space complexity, but slicing n indices will increase the time complexity by 2^n .

Slicing Technique

Slicing is a technique to reduce the space complexity of the tensor network by looping over a subset of indices. This effectively reduces the size of the tensor network inside the loop, and the space complexity can potentially be reduced. For example, in Figure 5, we slice the tensor network over

the index i . The label i is removed from the tensor network, at the cost of contraction multiple tensor networks.

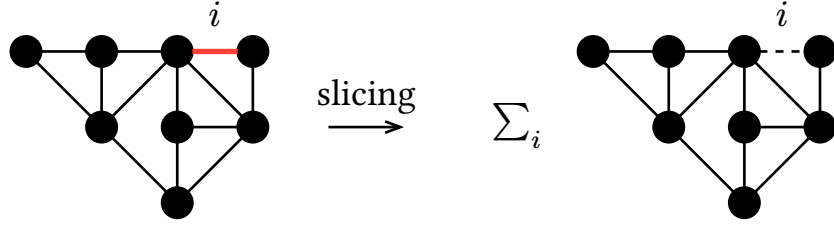


Figure 5: The slicing technique. The tensor network is sliced over the index i .

Continuing from the previous example, we can use the `slice_code` function to reduce the space complexity.

```
julia> sliced_code = slice_code(optcode, sizes,
TreeSASlicer(score=ScoreFunction(sc_target=cc.sc-3)));

julia> sliced_code.slicing
3-element Vector{Int64}:
 14
 76
 60

julia> contraction_complexity(sliced_code, sizes)
Time complexity: 2^17.800899899920303
Space complexity: 2^10.0
Read-write complexity: 2^17.199595668955244
```

The `slice_code` function takes three inputs: the `NestedEinsum` object, the tensor sizes, and the slicing strategy. Here, we use the `TreeSASlicer` with the `ScoreFunction` to reduce the space complexity by 3. The result type is `SlicedEinsum`, which contains a `slicing` field for storing the slice indices. After slicing, the space complexity is reduced by 3, while the time complexity is only slightly increased. The usage of `SlicedEinsum` is the same as the `NestedEinsum` object.

```
julia> @assert sliced_code(tensors...) ≈ optcode(tensors...)
```

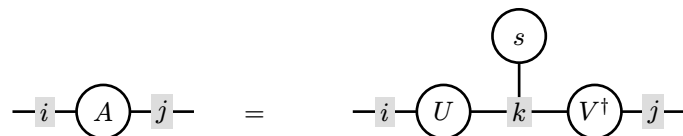
Data Compression and Tensor Decomposition

Tensor decomposition provides a systematic approach to data compression by representing high-dimensional objects as networks of smaller, more manageable tensors. This section explores how various decomposition strategies achieve different compression-computation trade-offs.

Singular Value Decomposition (SVD): Consider a complex matrix $A \in \mathbb{C}^{m \times n}$ with singular value decomposition:

$$A = USV^\dagger \quad (15)$$

where U and V are unitary matrices and S contains non-negative singular values. The tensor network representation of this decomposition reveals its essential structure:



The compression efficiency depends on the dimensions involved. Let $d_i = d \text{ im}(i)$, $d_j = d \text{ im}(j)$, and $d_k = d \text{ im}(k)$ denote the sizes of the respective indices.

Compression condition: Meaningful compression requires $d_k < \min(d_i, d_j)$ —the internal bond dimension must be smaller than both external dimensions.

Compression ratio: The storage reduction achieved is $\frac{d_i d_j}{d_k(d_i + d_j)}$, representing the ratio of original matrix elements to total decomposed elements.

CP Decomposition

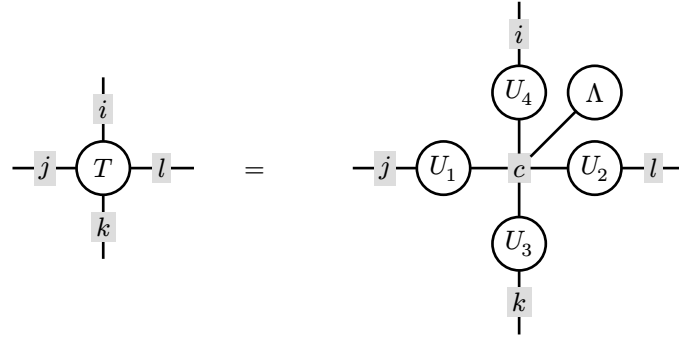
(Definition 8) (**CP Decomposition.**): The *Canonical Polyadic (CP) decomposition*, also known as CANDECOMP/PARAFAC, represents a tensor as a sum of rank-1 components. For an N -mode tensor $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_N}$, the CP decomposition is:

$$\mathcal{T}_{i_1, i_2, \dots, i_N} = \sum_{r=1}^R \lambda_r u_1^{(r)_{i_1}} u_2^{(r)_{i_2}} \dots u_N^{(r)_{i_N}} \quad (16)$$

where:

- R is the *CP rank* (number of rank-1 components)
- λ_r are optional scaling factors
- $u_k^r \in \mathbb{R}^{d_k}$ are factor vectors for the k -th mode and r -th component

This decomposition is particularly effective for tensors with inherent low-rank structure.



Compression analysis: For an N -mode tensor with mode dimensions $\{d_i\}_{i=1}^N$ and CP rank R :

- Original storage: $\prod_{i=1}^N d_i$ elements
- Decomposed storage: $R \sum_{i=1}^N d_i + R$ elements (including scaling factors)
- Compression ratio: $\frac{\prod_{i=1}^N d_i}{R(\sum_{i=1}^N d_i + 1)}$

For our rank-4 example: $\frac{d_i d_j d_k d_l}{R(d_i + d_j + d_k + d_l + 1)}$

Key advantage: Storage scales linearly with tensor order, avoiding the “curse of dimensionality.”

Tucker Decomposition

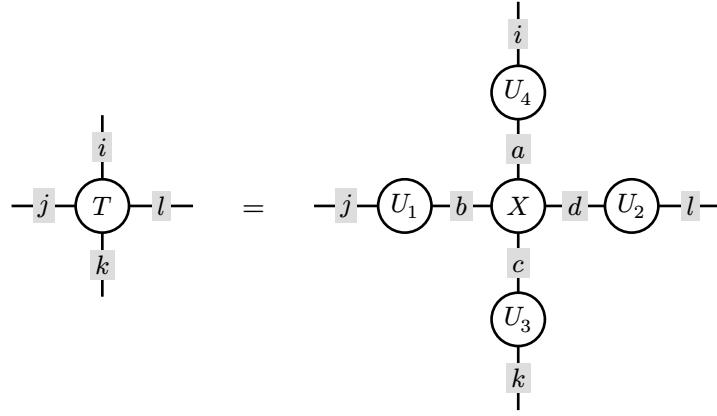
(Definition 9) (**Tucker Decomposition.**): The *Tucker decomposition* generalizes matrix SVD to higher-order tensors by decomposing each mode separately while retaining a dense core tensor. For an N -mode tensor $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_N}$, the Tucker decomposition is:

$$\mathcal{T}_{i_1, i_2, \dots, i_N} = \sum_{j_1=1}^{r_1} \sum_{j_2=1}^{r_2} \dots \sum_{j_N=1}^{r_N} \mathcal{X}_{j_1, j_2, \dots, j_N} \prod_{k=1}^N U_k^{(i_k, j_k)} \quad (17)$$

where:

- $\mathcal{X} \in \mathbb{R}^{r_1 \times r_2 \times \dots \times r_N}$ is the *core tensor* encoding mode interactions
- $U_k \in \mathbb{R}^{d_k \times r_k}$ are mode-wise factor matrices (typically orthogonal)
- r_k are the mode ranks (dimensions of the core tensor)

This decomposition provides more flexibility than CP decomposition at the cost of exponential core tensor growth with the number of modes.



The data compression ratio for Tucker decomposition is $\frac{\prod_{i=1}^N d_i}{\prod_{i=1}^N r_i + \sum_{i=1}^N d_i r_i}$, where d_i is the dimension of the i -th mode, N is the number of modes, and r_i is the dimension of the i -th core tensor mode. For the rank-4 case shown above, this becomes $\frac{d_i d_j d_k d_l}{r_a r_b r_c r_d + d_i r_a + d_j r_b + d_k r_c + d_l r_d}$.

The Tucker decomposition exhibits distinct computational characteristics that determine its practical applicability. Unlike CP decomposition, which imposes uniform rank constraints across all modes, Tucker decomposition permits independent rank selection for each tensor mode, enabling targeted compression strategies that exploit mode-specific structure. This flexibility allows the decomposition to capture complex multilinear dependencies that would require prohibitively high CP ranks.

However, this representational power comes at a fundamental computational cost. The core tensor X scales as $\prod_{i=1}^N r_i$, where r_i denotes the rank of the i -th mode and N is the tensor order. This exponential scaling in the number of modes—commonly termed the “curse of dimensionality”—severely constrains the method’s applicability to high-order tensors. For tensors of order $N > 6$, the core tensor typically becomes the dominant storage component, negating compression benefits.

Consequently, Tucker decomposition finds optimal application in moderate-order scenarios ($N \leq 4$) where the exponential core scaling remains computationally tractable while preserving the method’s superior representational capabilities.

Tensor Train

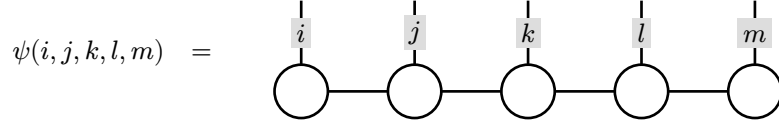
(Definition 10) (**Tensor Train (TT)**): *Tensor Train (TT)* is a specific tensor network architecture that represents high-dimensional tensors as a sequential chain of lower-rank tensors. For an N -mode tensor $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_N}$, the TT decomposition is:

$$\mathcal{T}_{i_1, i_2, \dots, i_N} = \sum_{\alpha_1, \dots, \alpha_{N-1}} A_1^{(i_1)_{\alpha_1}} A_2^{(i_2)_{\alpha_1, \alpha_2}} \dots A_N^{(i_N)_{\alpha_{N-1}}} \quad (18)$$

where:

- $A_1^{i_1} \in \mathbb{R}^{r_1}$ is the first tensor (vector)
- $A_k^{i_k} \in \mathbb{R}^{r_{k-1} \times r_k}$ are intermediate tensors for $k = 2, \dots, N-1$
- $A_N^{i_N} \in \mathbb{R}^{r_{N-1}}$ is the last tensor (vector)
- r_k are the *bond dimensions* (ranks of virtual indices)

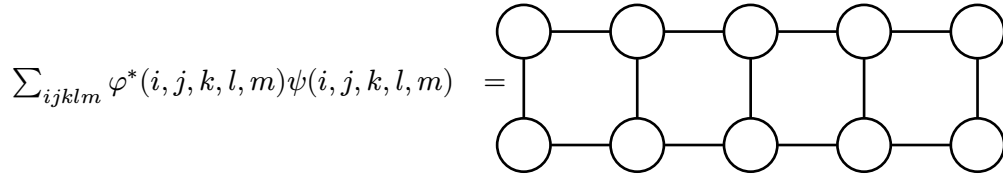
The storage requirement scales as $O(\sum_{k=1}^N d_k r_{k-1} r_k)$ with $r_0 = r_N = 1$, providing efficient compression for high-dimensional tensors.



This architecture represents a high-dimensional tensor using a compact one-dimensional chain structure. With bond dimension χ (the size of virtual indices connecting adjacent tensors), the storage requirement scales as $O(d\chi^2 L)$, where d is the physical dimension and L is the chain length. This yields a compression ratio of $O(\frac{d^L}{\chi^{2L}})$ compared to the full tensor.

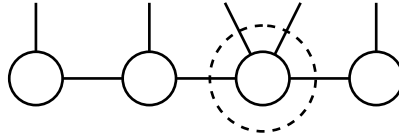
The tensor train format offers several computational advantages:

1. Efficient inner products. Computing overlaps between two tensor trains requires only local contractions:

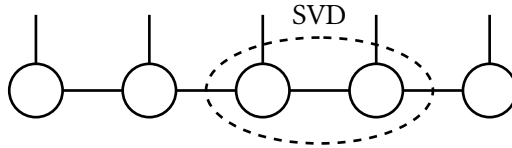


2. Polynomial-time compression. Unlike many tensor decompositions, tensor trains admit efficient compression through iterative sweeping algorithms that alternately apply:

1. Contract two tensors



2. Tensor factorization



The factorization is usually done by first reshaping the tensor into a matrix and then applying singular value decomposition. By eliminating small singular values, the bond dimension can be

reduced. Easy to compress is a feature of all loopless tensor networks, including the tensor train. In the following example, we show a uniform state can be represented as a tensor train of rank 1.

```
julia> uniform_state(n) = fill(sqrt(1/2^n), 2^n);

julia> L, M, R = fill(sqrt(0.5), 2, 1), fill(sqrt(0.5), 1, 2, 1),
fill(sqrt(0.5), 1, 2);

julia> @assert ein"ia,ajb,bkc,cld,dm->ijklm"(L, M, M, M, R) ≈ uniform_state(5)
```

Example: High-Dimensional Tensor Compression with Tensor Train

This example demonstrates practical tensor train decomposition for compressing exponentially large tensors into manageable representations. We'll compress a tensor with $2^{20} \approx 1$ million elements using polynomial storage.

Data structure: We define a Matrix Product State (MPS) to store the tensor train:

```
using OMEinsum, LinearAlgebra

# MPS represents a tensor train decomposition
struct MPS{T}
    tensors::Vector{Array{T, 3}} # Each tensor has (left_bond, physical,
    right_bond)
end
```

Decomposition algorithm: The tensor train decomposition proceeds through iterative SVD sweeps:

```
# Compress tensor using iterative SVD-based tensor train decomposition
function tensor_train_decomposition(tensor::AbstractArray, max_rank::Int;
    atol=1e-6)
    dims = size(tensor)
    n = length(dims)
    tensors = Array{Float64, 3}[]

    # Initialize left bond dimension
    left_bond = 1
    current_tensor = reshape(tensor, dims[1], :) # Matricize first mode

    # Sweep through all modes except the last
    for i in 1:(n-1)
        # SVD with rank truncation
        U, S, V, new_rank = truncated_svd(current_tensor, max_rank, atol)

        # Store current tensor with proper bond structure
        push!(tensors, reshape(U, (left_bond, dims[i], new_rank)))

        # Prepare remainder for next iteration
        remainder = Diagonal(S) * V'
        current_tensor = reshape(remainder, new_rank * dims[i+1], :)
```

```

        left_bond = new_rank
    end

    # Final tensor has right bond dimension 1
    push!(tensors, reshape(current_tensor, (left_bond, dims[n], 1)))
    return MPS(tensors)
end

```

Algorithm insight: Each iteration performs SVD to optimally separate the current mode from all remaining modes, then truncates small singular values to control bond dimensions. This greedy approach often achieves near-optimal compression for many practical tensors.

```

function truncated_svd(matrix::AbstractArray, max_rank::Int, atol)
    U, S, V = svd(matrix)

    # Determine truncation rank: respect both rank limit and error
    tolerance
    significant_values = S .> atol
    effective_rank = min(max_rank, sum(significant_values))

    # Return truncated factors
    return U[:, 1:effective_rank], S[1:effective_rank], V[:,
1:effective_rank], effective_rank
end

```

Truncation strategy: The algorithm balances two competing objectives by maintaining bond dimensions at computationally manageable levels while retaining singular values above the specified error threshold. This dual constraint ensures both computational tractability and approximation accuracy.

To recover the tensor, we construct the matrix product state, we construct the tensor network topology and

```

# Function to contract the TT cores to reconstruct the tensor
function contract(mps::MPS)
    n = length(mps.tensors)
    code = EinCode([2i-1, 2i, 2i+1] for i in 1:n], Int[2i for i in 1:n])
    size_dict = OMEinsum.get_size_dict(code.ixs, mps.tensors)
    optcode = optimize_code(code, size_dict, GreedyMethod())
    return optcode(mps.tensors...)
end

```

Practical demonstration: We compress a uniform tensor with $2^{20} \approx 1$ million elements:

```

# Create a uniform tensor (all entries equal to 1)
tensor = ones(Float64, fill(2, 20)...) # Shape: (2,2,2,...,2) with 20
modes

```

```

# Perform tensor train decomposition
mps = tensor_train_decomposition(tensor, max_rank=5)
reconstructed = contract(mps)

# Analyze compression performance
relative_error = norm(tensor - reconstructed) / norm(tensor)
# Result: ~5e-12 (nearly perfect reconstruction)

original_storage = prod(size(tensor)) # 2^20 = 1,048,576 elements
compressed_storage = sum(prod(size(core)) for core in mps.tensors)
compression_ratio = original_storage / compressed_storage
# Result: ~26,214 (massive compression!)

```

Compression analysis: The uniform tensor has remarkable structure—its tensor train rank is exactly 1, enabling each core tensor to store only 2 elements. This demonstrates how tensor networks can exploit hidden low-rank structure for exponential compression gains.

Automatic Differentiation

Backpropagation constitutes a fundamental machine learning technique for computing gradients of loss functions $\mathcal{L}$ with respect to model parameters. Its foundation rests on the **backward rule**, which efficiently propagates adjoint information through computational graphs. The adjoint of a variable a is defined as $\bar{a} = \frac{\partial \mathcal{L}}{\partial a}$, representing the sensitivity of the loss to changes in that variable.

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with input x and known adjoint of the output $\bar{y}$, the backward rule computes the adjoint of the input as:

$$\bar{x} = \frac{\partial f^T}{\partial x} \bar{y} \quad (19)$$

This process efficiently propagates gradient information backward through the network, enabling optimization of complex models.

For matrix multiplication $C = AB$, the backward rule yields:

$$\bar{A} = \bar{C}B^T, \quad \bar{B} = A^T\bar{C} \quad (20)$$

This rule exemplifies the remarkable efficiency of backpropagation. While the full Jacobian matrix would contain $O(n^4)$ elements, the backward computation requires only $O(n^3)$ matrix operations—the same complexity as the forward pass. This efficiency breakthrough enables practical optimization of complex models and underlies the success of modern deep learning. Tensor network contraction generalizes matrix multiplication while preserving differentiation efficiency. We represent a tensor network as the triple $(\Lambda, \mathcal{T}, \sigma_Y)$:

$$Y = \text{contract}(\Lambda, \mathcal{T}, \sigma_Y) \quad (21)$$

where Λ contains all tensor indices, $\mathcal{T}$ holds the tensor collection, and $\sigma_Y \subset \Lambda$ specifies output indices.

The backward rule for computing input tensor gradients follows naturally:

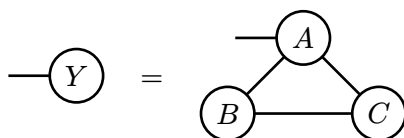
$$\bar{X} = \text{contract}(\Lambda, (\mathcal{T} \setminus \{X\}) \cup \{\bar{Y}\}, \sigma_X) \quad (22)$$

where $\mathcal{T} \setminus \{X\}$ represents the tensor set excluding X , and σ_X denotes X 's indices.

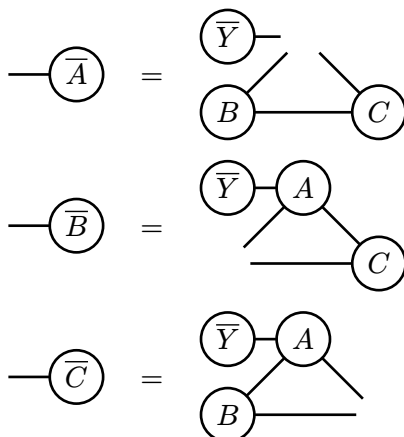
While naive implementation would require separate network contractions for each input (linear overhead), sophisticated binary contraction trees reduce this to constant overhead. Modern automatic differentiation achieves gradient computation at approximately twice the forward pass cost—remarkable efficiency considering the inherent complexity of multilinear operations.

Example: Backward rule for tensor network contraction

Consider the tensor network contraction: $Y = \text{ein}''\text{a}ij,jk,ki\text{->a}''(A, B, C)$, where A, B, C are tensors labeled by $(a, i, j), (j, k), (k, i)$ respectively. Diagrammatically, the forward contraction is given by:



The backward rule is given by:



Quiz: If the forward contraction specified with a binary contraction order: $Y = \text{ein}''(aij,jk),ki\text{->a}''(A, B, C)$, how are gradients computed in the backward propagation?

In OMEinsum, the backward rule of einsum has already been ported to ChainRulesCore, which can be directly used in Zygote and Flux. It also implements a

```
julia> gradients = cost_and_gradient(optcode, (tensors...,));
```

The returned gradients is a vector of arrays, each of which is an adjoint of an input tensor.

Quantum Circuit Simulation

Quantum States and Quantum Gates

(Definition 11) (**Quantum State.**): A *quantum state* is a mathematical representation of a quantum system described by a vector in a complex Hilbert space. For an n -qubit system, the quantum state $|\psi\rangle$ belongs to the Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, where each qubit contributes a 2-dimensional complex vector space. The state can be written as:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \quad (23)$$

where $\alpha_i \in \mathbb{C}$ are complex amplitudes satisfying the normalization condition $\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$, and $|i\rangle$ denotes the computational basis states.

(Definition 12) (**Quantum Gate.**): A *quantum gate* is a unitary transformation that acts on quantum states, represented mathematically by a unitary matrix U satisfying $UU^\dagger = I$. For single-qubit gates, $U \in \mathbb{C}^{2 \times 2}$, while k -qubit gates have $U \in \mathbb{C}^{2^k \times 2^k}$. The gate transforms a quantum state according to:

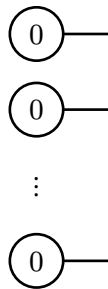
$$|\psi\rangle \mapsto U |\psi\rangle \quad (24)$$

In tensor network representation, quantum gates become tensors where:

- Single-qubit gates are rank-2 tensors (matrices)
- Multi-qubit gates are higher-rank tensors with indices corresponding to input and output qubits

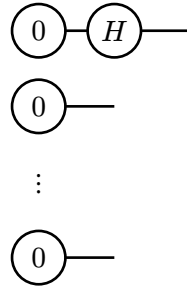
Quantum circuits map naturally onto tensor networks through this correspondence, transforming quantum circuit simulation into tensor network contraction problems.

Initial state representation: A quantum system initialized to $|0\rangle^{\otimes n}$ (the n -fold tensor product of computational zero states) decomposes as a product of independent single-qubit states:

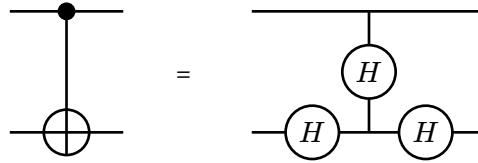


where each $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ state appears as a rank-1 tensor in the network.

Gate representation: Single-qubit gates correspond to rank-2 tensors (matrices) that transform individual qubits. For example, applying a Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ to the first qubit creates the tensor network:



Multi-qubit gates: Two-qubit and multi-qubit gates create richer tensor network structures by introducing shared virtual indices between previously independent qubits. The CNOT gate, which generates entanglement between qubits, admits the tensor network decomposition:

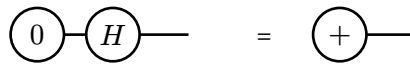


This decomposition (up to normalization) reveals a fundamental insight: two-qubit gates create **entanglement** by establishing shared virtual bonds between previously independent qubits. These virtual indices carry the quantum correlations that classical systems cannot efficiently represent.

Useful Circuit Identities

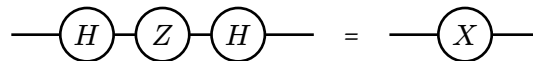
The graphical nature of tensor networks renders quantum circuit identities visually apparent, often revealing why certain simplifications work at an intuitive level. These identities become powerful tools for circuit optimization and theoretical analysis.

Identity 1: Hadamard basis transformation



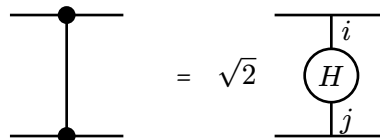
The Hadamard gate transforms the computational basis state $|0\rangle$ into the superposition state $|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$. Graphically, this substitution can be made wherever the pattern appears.

Identity 2: Basis rotation under conjugation



The conjugation identity $HZH = X$ reflects a deeper principle: the Hadamard gate rotates between the Z and X measurement bases. This basis transformation underlies many quantum algorithms, from quantum teleportation to variational quantum eigensolvers.

Identity 3: Symmetric two-qubit gates



The controlled-Z gate (CZ) admits a remarkably simple tensor network representation as a single rank-4 tensor. This symmetric implementation reveals that CZ is its own inverse and demonstrates how entangling gates create shared virtual indices.

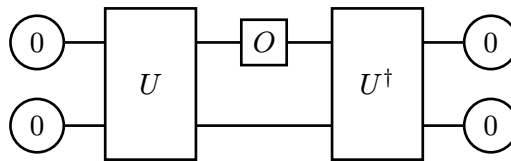
Verification: We can confirm this identity using OMEinsum:

```
julia> reshape(ein"ij->ijij"([1 1; 1 -1]), 4, 4)
4×4 Matrix{Int64}:
 1  0  0  0
 0  1  0  0
 0  0  1  0
 0  0  0 -1
```

Expectation Values

Computing quantum expectation values $\langle \psi | O | \psi \rangle$ requires working with both the quantum state and its complex conjugate, leading to distinctive “sandwich” tensor network patterns. This structure naturally emerges from the Born rule for quantum measurements.

Mathematical setup: For a quantum state $|\psi\rangle = U|0^n\rangle$ prepared by unitary circuit U and observable O , the expectation value becomes:



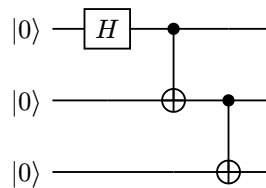
Tensor network structure: The resulting “sandwich” pattern represents the quantum mechanical formula $\langle 0^n | U^\dagger O U | 0^n \rangle$. This structure has several important features:

- The observable O sits between the forward evolution U and backward evolution $U^\dagger$
- Initial and final states are identical computational basis states
- The contraction computes a scalar expectation value

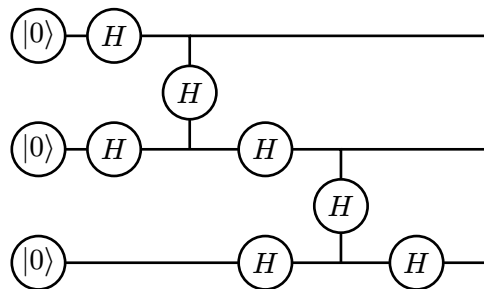
This pattern appears throughout quantum algorithms and forms the basis for variational quantum computing approaches.

Example: GHZ state preparation circuit

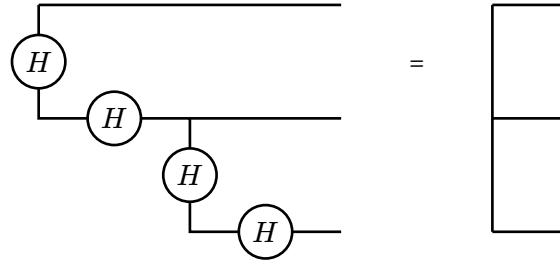
Consider a 3-qubit quantum circuit that prepares the GHZ state $|\text{GHZ}\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$. The quantum circuit generating this state is shown below:



The corresponding tensor network diagram is:



which can be simplified to



Question: How to compute $\langle \text{GHZ} | O | \text{GHZ} \rangle$ and what is the complexity?

Example: Hadamard test

(*Theorem 2*) (**Hadamard Test.**): The *Hadamard test* is a quantum algorithm that estimates the expectation value of a unitary operator U with respect to a quantum state $|\psi\rangle$ using a single ancilla qubit. The algorithm measures the expectation value:

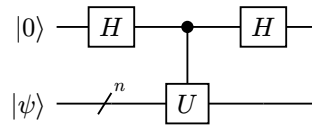
$$\langle \psi | U | \psi \rangle = \langle Z \rangle_{\text{ancilla}} \quad (25)$$

where the measurement is performed on the ancilla qubit in the computational basis after applying the Hadamard test circuit.

Specifically, measuring the ancilla qubit in the Z -basis yields:

- $\langle Z \rangle_{\text{ancilla}} = \text{Re}(\langle \psi | U | \psi \rangle)$ for the standard Hadamard test
- $\langle X \rangle_{\text{ancilla}} = \text{Im}(\langle \psi | U | \psi \rangle)$ for the modified Hadamard test (with additional S gate)

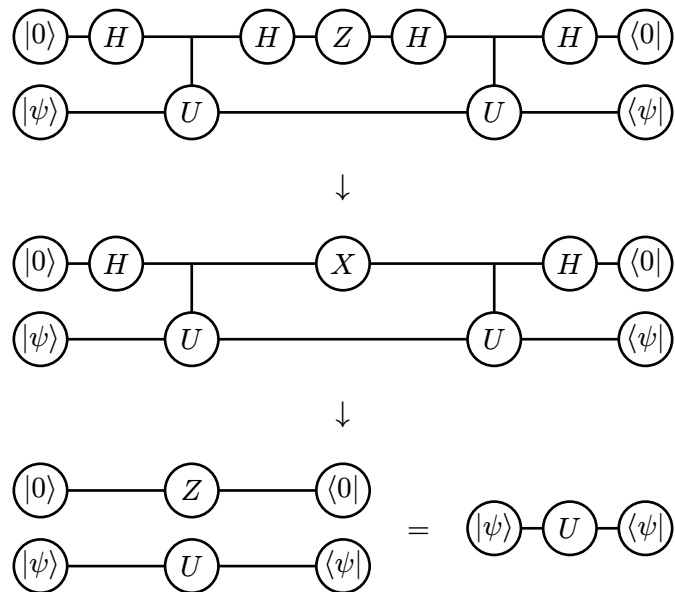
The Hadamard test circuit is shown below:



The expectation value of Z on the first qubit is given by

$$\langle Z \rangle = \text{Re}(\langle \psi | U | \psi \rangle) \quad (26)$$

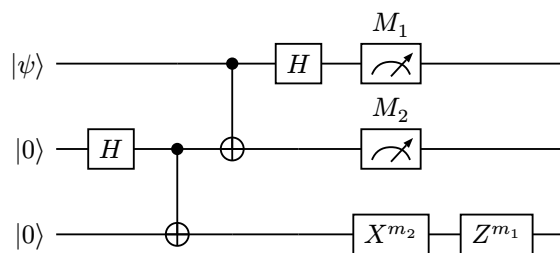
The corresponding tensor network representation is:



Example: Quantum teleportation

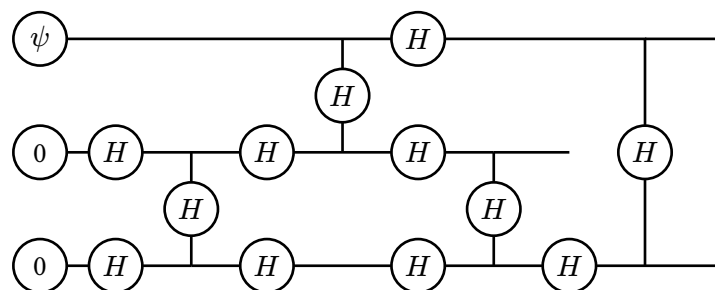
Teleportation transmits an unknown state $|\psi\rangle$ from Alice to Bob using a shared Bell pair and two classical bits. The steps are: (1) prepare a Bell pair on qubits 2–3, (2) perform a Bell-basis measurement on qubits 1–2, (3) apply Pauli corrections $Z^{m_1} X^{m_2}$ on qubit 3 according to outcomes (m_1, m_2) .

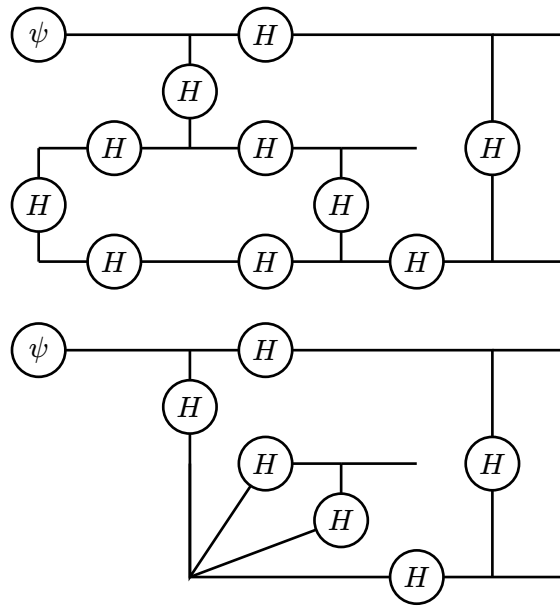
Circuit



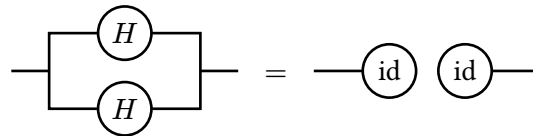
Tensor-network diagram and simplification

The circuit maps to a tensor network where the Bell pair is a rank-2 tensor, gates are rank-4 tensors, and measurements are projectors. Up to Pauli frame corrections, the network reduces to an identity wire from Alice's input to Bob's output.

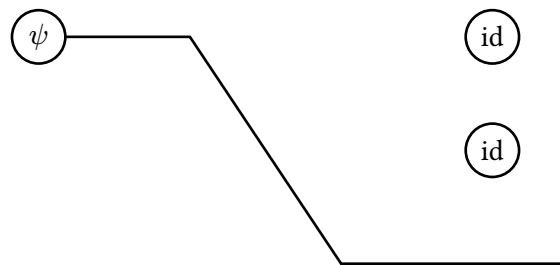




Here, we use the following identity:



Then we have



Quantum channel simulation

Quantum channels represent the evolution of open quantum systems, capturing both unitary evolution and decoherence effects. In tensor network simulations, these are implemented through the Kraus operator formalism and density matrix evolution.

Kraus operators

(Definition 13) (**Kraus Operators**): *Kraus operators* provide a mathematical representation of quantum channels describing the evolution of open quantum systems. A quantum channel $\mathcal{E}$ acting on density matrices is characterized by a set of Kraus operators $\{K_i\}$ such that:

$$\mathcal{E}(\rho) = \sum_i K_i \rho K_i^\dagger \quad (27)$$

where ρ is any density matrix. The Kraus operators must satisfy the *completeness relation*:

$$\sum_i K_i^\dagger K_i = I \quad (28)$$

This representation ensures the map is *completely positive* (CP) and *trace-preserving* (TP), meaning it preserves the positivity and normalization of density matrices, corresponding to valid quantum evolution.

This formalism captures both unitary evolution and decoherence effects, enabling the description of realistic quantum systems interacting with their environment.

This formalism allows us to describe various noise processes:

Amplitude damping

Models energy loss processes with Kraus operators:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \quad (29)$$

Phase damping

Models pure dephasing with:

$$K_0 = \sqrt{1 - \frac{\gamma}{2}} I, \quad K_1 = \sqrt{\frac{\gamma}{2}} Z \quad (30)$$

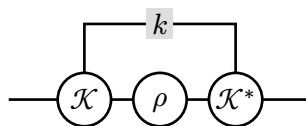
Depolarizing channel

The most commonly used noise model, with Kraus operators:

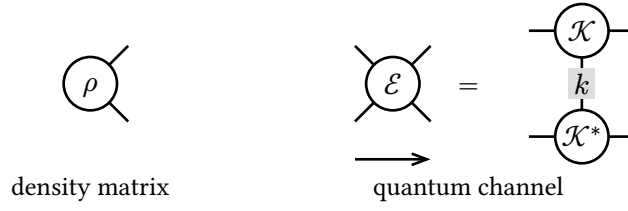
$$K_0 = \sqrt{1 - 3\frac{p}{4}} I, \quad K_1 = \sqrt{\frac{p}{4}} X, \quad K_2 = \sqrt{\frac{p}{4}} Y, \quad K_3 = \sqrt{\frac{p}{4}} Z \quad (31)$$

Tensor network representation of channels

Consider applying a Kraus channel $\mathcal{E}$ to a density matrix ρ . The result can be diagrammatically represented as



Sometimes, we use the superoperator representation, which corresponds to the contracted Kraus channels



For example, the superoperator representation of the depolarizing channel is

```
julia> using OMEinsum, Yao, SymEngine

julia> p = Basic(:p) # define a symbolic variables
p

julia> K = cat(sqrt(1-3p/4) * Matrix{Basic}(I2), sqrt(p/4) * Matrix{Basic}(X),
sqrt(p/4) * Matrix{Basic}(Y), sqrt(p/4) * Matrix{Basic}(Z); dims=3);

julia> superop_dep = reshape(ein"abk,cdk->acbd"(K, conj(K)), 4, 4)
4×4 Matrix{Basic}:
 1 + (-1/2)*p      0      0      (1/2)*p
      0 1 - p      0      0
      0 0 1 - p      0
 (1/2)*p      0      0 1 + (-1/2)*p
```

Pauli transfer matrix formulation

The PTM formalism provides a powerful framework for classical simulation of noisy quantum circuits. In this representation, the normalized Pauli basis $\mathbb{P} = \frac{\{I, X, Y, Z\}}{\sqrt{2}}$ forms an orthonormal basis for the operator space, where single-qubit quantum states become vectors $|\rho\rangle\rangle_P$ with components:

$$(|\rho\rangle\rangle_P)_i = \text{tr}(\rho P_i), \quad P_i \in \mathbb{P} \quad (32)$$

Let us denote the superoperator (vectorized) representation of density matrix ρ as $|\rho\rangle\rangle$. The Pauli basis representation corresponds to the following basis transformation:

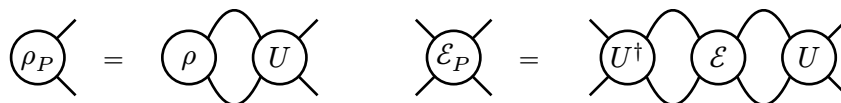
$$|\rho\rangle\rangle_P = U|\rho\rangle\rangle$$

$$U = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}} \\ 0 & \frac{1}{\sqrt{2}} & \frac{-i}{\sqrt{2}} & 0 \\ 0 & \frac{1}{\sqrt{2}} & \frac{i}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & 0 & 0 & \frac{-1}{\sqrt{2}} \end{pmatrix} \quad (33)$$

The four columns correspond to the vectorized and normalized Pauli matrices. Then, we also apply this basis transformation to the quantum channel $\mathcal{E}$:

$$\mathcal{E}_P = U \mathcal{E} U^\dagger \quad (34)$$

Diagrammatically, this transformation is



For the depolarizing channel, the Pauli basis representation can be obtained by:

```
julia> U = Basic[1 0 0 1; 0 1 -im 0; 0 1 im 0; 1 0 0 -1] / sqrt(Basic(2));

julia> pauli_dep = SymEngine.expand.(U * superop_dep * U')
4×4 Matrix{Basic}:
 1      0      0      0
 0 1 - p      0      0
 0      0 1 - p      0
 0      0      0 1 - p
```

It is a diagonal matrix $\mathcal{D}_P = \text{diag}(1, 1 - p, 1 - p, 1 - p)$, enabling efficient multi-qubit simulation via tensor decomposition:

$$\mathcal{D}_P = (1 - p)I + p|0\rangle\langle 0| \quad (35)$$

Or diagrammatically,

$$\text{Diagram of } \mathcal{D}_P \text{ (circle with } \mathcal{D}_P \text{ and four legs)} = (1 - p) \text{Diagram of } I \text{ (two parallel horizontal lines)} + p \text{Diagram of } |0\rangle\langle 0| \text{ (two circles with } |0\rangle \text{ and } \langle 0| \text{ on the legs)}$$

In the path-integral point of view, we either pick the first term or the second term in a single path. The first term has the power of damping the amplitude of states, while the second term has rank 1, and can be used to truncate the tensor network. As a consequence, quantum circuits with finite depolarizing noise can be simulated in polynomial time[12], [13].

Quantum Error Correction

Quantum error correction (QEC) addresses a fundamental challenge in quantum computing: protecting fragile quantum information from inevitable errors caused by environmental decoherence, imperfect control systems, and gate imperfections[14], [15], [16].

Core principle: QEC operates by encoding logical quantum information into a larger physical Hilbert space with built-in redundancy. This encoding creates “error syndromes” that reveal error locations without destroying the logical information—a delicate balance unique to quantum mechanics.

Stabilizer formalism: Most practical QEC schemes are described using the elegant stabilizer formalism, which characterizes quantum codes through their symmetries rather than explicit state vectors.

Stabilizers and Quantum Codes

The stabilizer formalism provides an algebraic framework for constructing and analyzing quantum error-correcting codes through group theory.

(Definition 14) (**Pauli Group and Stabilizer Group.**): The n -qubit Pauli group is the group generated by tensor products of single-qubit Pauli matrices[17]:

$$\mathcal{P}_n = \{\pm 1, \pm i\} \times \{I, X, Y, Z\}^{\otimes n} \quad (36)$$

Elements of this group are called *Pauli operators* or *Pauli strings*.

A *stabilizer group* $\mathcal{S}$ is an Abelian subgroup of $\mathcal{P}_n$ that contains only operators with eigenvalue $+1$ on all code states. The requirement of commutativity ensures that stabilizer measurements are compatible and do not disturb each other.

Generator structure: Any stabilizer group can be specified by a set of independent generators $\{S_a\}_{a=1, \dots, m}$:

$$\mathcal{S} = \langle S_1, S_2, \dots, S_m \rangle \quad (37)$$

Code space definition: The quantum code space consists of all states that are $+1$ eigenvectors of every stabilizer operator. This space contains the protected logical information.

Error detection: Measuring the stabilizer generators reveals whether errors have occurred:

- All generators yield $+1$: state remains in code space
- Any generator yields -1 : error detected, creating an “error syndrome”

Key insight: Stabilizer measurements preserve quantum information because they project onto subspaces (code space or error spaces) rather than classical bit values. The logical quantum state remains coherent within these subspaces.

(Definition 15) (**Quantum Code.**): An $[[n, k, d]]$ quantum code encodes k logical qubits into n physical qubits with minimum distance d . The parameters have precise meanings:

- n : number of physical qubits required
- k : number of logical qubits protected
- d : minimum weight of Pauli operators that can transform one codeword into a different codeword

The distance d determines error-correcting capability: the code can detect up to $d - 1$ errors and correct up to $\lfloor \frac{d-1}{2} \rfloor$ errors.

Code specification: An $[[n, k, d]]$ quantum code is typically specified by providing $n - k$ independent stabilizer generators, with the remaining degrees of freedom corresponding to the k logical qubits.

(Definition 16) (**CSS Code**[16], [18], [19].): A quantum stabilizer code is called a *Calderbank-Shor-Steane (CSS) code* if its stabilizer group can be generated entirely by Pauli operators containing only X or only Z (no Y operators):

$$\mathcal{S} = \langle S_a \rangle_{a=1, \dots, n-k}, \quad \text{where } S_a \in \{I, X\}^{\otimes n} \cup \{I, Z\}^{\otimes n} \quad (38)$$

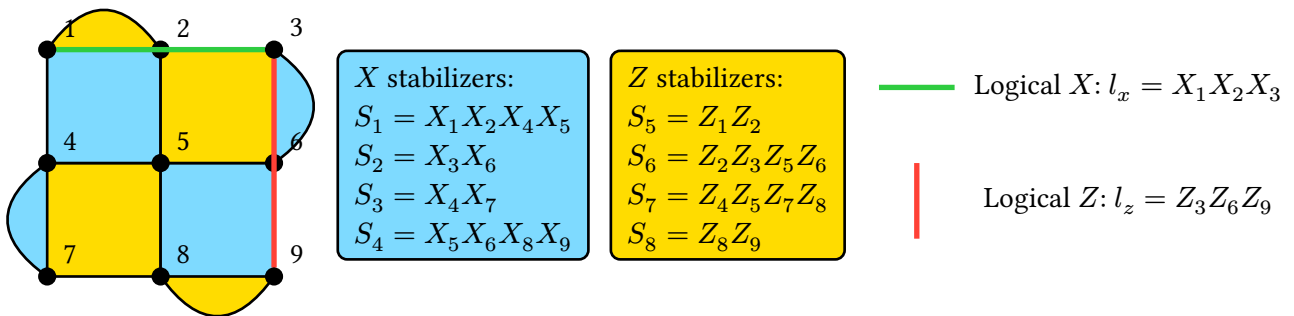
This structure enables classical error correction techniques to be applied separately to bit-flip (X) and phase-flip (Z) errors.

Practical significance: CSS codes represent a large and important class of quantum error-correcting codes that can be constructed using classical error correction principles. Many of the most practical quantum codes, including surface codes and color codes, belong to this family due to their relatively simple implementation requirements.

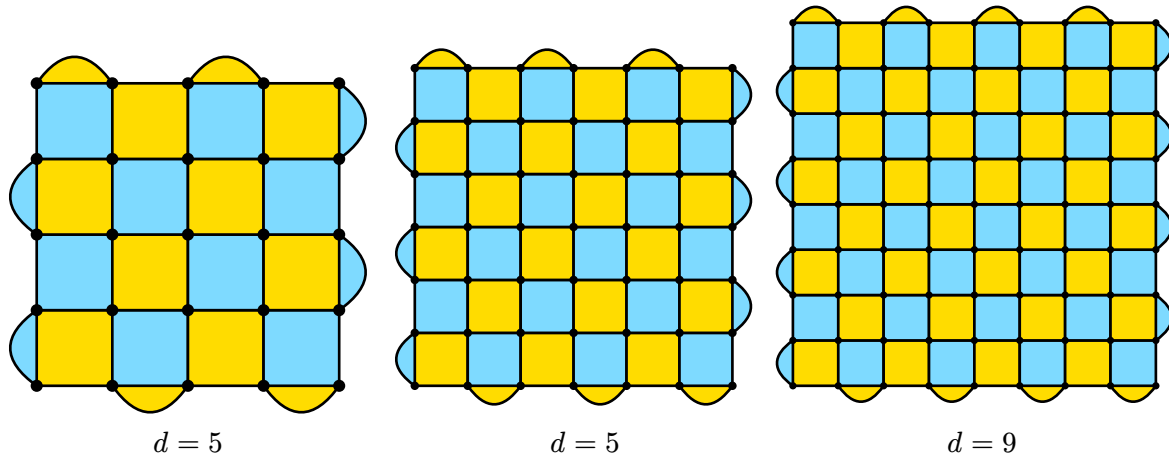
Surface code

The surface code [20], [21] is a prominent example of a topological quantum error-correcting code, defined on a two-dimensional lattice of qubits arranged in a grid. Each plaquette (face) of the lattice is associated with a stabilizer operator, which acts on the qubits at the corners of the plaquette.

There are two types of stabilizers: X -type (acting with Pauli X operators) and Z -type (acting with Pauli Z operators), typically arranged in a checkerboard pattern. Here is an example of $[[9, 1, 3]]$ surface code. The stabilizers are shown in the figure. The logical operator $X_1 X_2 X_3$ and $Z_3 Z_6 Z_9$ commute with all stabilizers and do not belong to the stabilizer group. The length of them is exactly the distance of the code.



Also we can have different sizes of the surface code.



Decoding problem

If some Pauli errors happened, some of the stabilizers will be anti-commute with the errors. When we measure them we usually call the measurement outcome of the all stabilizers as syndrome. The decoding problem is given the syndrome, find the probable error pattern that is consistent with the syndrome.

(Definition 17) (**MLE Problem.**): The most-likely error (MLE) problem is given the syndrome, find the most probable error pattern that is consistent with the syndrome.

$$\begin{aligned} \underset{e}{\operatorname{argmax}} p(e) \\ \text{s.t. } H(e) = s \end{aligned} \quad (39)$$

where $H(e)$ is the syndrome of the error pattern e .

For a given error, applying any stabilizer to it leaves the syndrome unchanged. All such errors within the same degenerate class have an equivalent effect on the logical information. Thus, a better decoding approach than MLE is MLD (Maximum Likelihood Decoding), which directly determines the most probable logical effect of the error rather than the exact physical error.

(Definition 18) (**MLD Problem.**): The maximum likelihood decoding (MLD) problem is given the syndrome, find the most probable logical state by summing over all the error patterns that belong to the same degenerate class and are consistent with the syndrome.

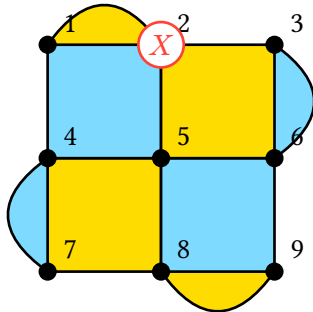
$$\underset{l}{\operatorname{argmax}} p(l) = \underset{l}{\operatorname{argmax}} \sum_{\substack{L(e)=l \\ H(e)=s}} p(e) \quad (40)$$

where $H(e)$ is the syndrome of the error pattern e , $L(e)$ is the logical information of the error pattern e .

As shown, the MLD decoding problem involves summing over all error patterns within the same degenerate class that match the observed syndrome. This structure naturally lends itself to tensor network contraction methods. In the following section, we will introduce tensor network-based MLD decoder. Now we will give an example of the decoding problem.

Example: Decoding problem

Suppose there is an X error on the qubit 2. The decoding process is to find the most probable error pattern that is consistent with the syndrome.



X stabilizers:

$X_1 X_2 X_4 X_5$
 $X_3 X_6$
 $X_4 X_7$
 $X_5 X_6 X_8 X_9$

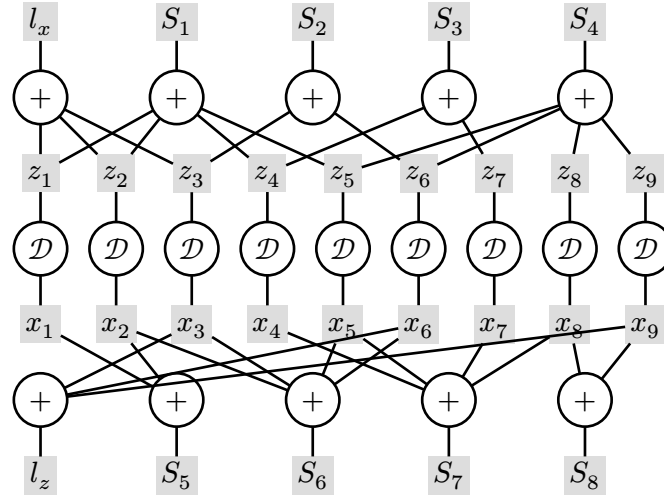
Z stabilizers:

$Z_1 Z_2$
 $Z_2 Z_3 Z_5 Z_6$
 $Z_4 Z_5 Z_7 Z_8$
 $Z_8 Z_9$

Only stabilizer $Z_1 Z_2$ and $Z_2 Z_3 Z_5 Z_6$ is anti-commute with the error. So if we measure all the stabilizers, we will get six $+1$ and two -1 . Based on this syndrome, decoders will try to find the most probable error pattern or logical state.

Tensor network decoder

Here we directly give the tensor network representation[5], [22] of the decoding problem. The dimension of the variables is 2.



In the middle of the figure, we have 9 tensors represent the depolarizing channel acts on the physical qubits.

$$\mathcal{D} = \begin{pmatrix} p_I & p_Z \\ p_X & p_Y \end{pmatrix}$$

Depolarizing Channel:

$$\mathcal{D}(\rho) = (1 - p_X - p_Y - p_Z)\rho + p_X X\rho X + p_Y Y\rho Y + p_Z Z\rho Z$$

The variables connected to the depolarizing channel represent Boolean variables indicating X or Z errors on the physical qubits. The $+$ tensors are the parity tensors.

$$\begin{array}{c} j_1 \\ | \\ \bigcirc + \\ / \quad \backslash \\ j_k \quad \dots \quad j_3 \end{array} \quad T(j_1, j_2, j_3, \dots, j_k) = \begin{cases} 1 & \text{if } j_1 + j_2 + \dots + j_k \text{ is even} \\ 0 & \text{if } j_1 + j_2 + \dots + j_k \text{ is odd} \end{cases}$$

$$j_1, j_2, j_3, \dots, j_k \in \{0, 1\}$$

The stabilizer variables are fixed to the measured syndrome values, while the logical variables represent the marginal probabilities computed via tensor network contraction. After contracting such a tensor network, we can get the marginal probability of the logical variables.

Bibliography

- [1] C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [2] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*, vol. 4, no. 4. Springer, 2006.
- [3] M. R. Villescas, J.-G. Liu, P. W. Wijnings, S. Stuijk, and H. Corporaal, “Scaling Probabilistic Inference Through Message Contraction Optimization,” in *2023 Congress in Computer Science*,

Computer Engineering, & Applied Computing (CSCE), Jul. 2023, pp. 123–130. doi: [10.1109/CSCE60160.2023.00025](https://doi.org/10.1109/CSCE60160.2023.00025).

- [4] I. L. Markov and Y. Shi, “Simulating Quantum Computation by Contracting Tensor Networks,” *SIAM Journal on Computing*, vol. 38, no. 3, pp. 963–981, Jan. 2008, doi: [10.1137/050644756](https://doi.org/10.1137/050644756).
- [5] C. Piveteau, C. T. Chubb, and J. M. Renes, “Tensor-Network Decoding Beyond 2D,” *PRX Quantum*, vol. 5, no. 4, p. 40303, Oct. 2024, doi: [10.1103/PRXQuantum.5.040303](https://doi.org/10.1103/PRXQuantum.5.040303).
- [6] Y. Qing, K. Li, P.-F. Zhou, and S.-J. Ran, “Compressing Neural Network by Tensor Network with Exponentially Fewer Variational Parameters,” no. arXiv:2305.06058. arXiv, May 2024. doi: [10.48550/arXiv.2305.06058](https://doi.org/10.48550/arXiv.2305.06058).
- [7] J. Haegeman, C. Lubich, I. Oseledets, B. Vandereycken, and F. Verstraete, “Unifying Time Evolution and Optimization with Matrix Product States,” *Physical Review B*, 2016, doi: [10.1103/PhysRevB.94.165116](https://doi.org/10.1103/PhysRevB.94.165116).
- [8] J. Gray and S. Kourtis, “Hyper-optimized tensor network contraction,” *Quantum*, vol. 5, p. 410, Mar. 2021, doi: [10.22331/q-2021-03-15-410](https://doi.org/10.22331/q-2021-03-15-410).
- [9] M. Roa-Villescas, X. Gao, S. Stuijk, H. Corporaal, and J.-G. Liu, “Probabilistic Inference in the Era of Tensor Networks and Differential Programming,” *Physical Review Research*, vol. 6, no. 3, p. 33261, Sep. 2024, doi: [10.1103/PhysRevResearch.6.033261](https://doi.org/10.1103/PhysRevResearch.6.033261).
- [10] F. V. Fomin and K. Høie, “Pathwidth of Cubic Graphs and Exact Algorithms,” *Information Processing Letters*, vol. 97, no. 5, pp. 191–196, 2006, doi: [10.1016/j.ipl.2005.10.012](https://doi.org/10.1016/j.ipl.2005.10.012).
- [11] G. Kalachev, P. Panteleev, and M.-H. Yung, “Recursive Multi-Tensor Contraction for XEB Verification of Quantum Circuits.” 2021.
- [12] X. Gao and L. Duan, “Efficient Classical Simulation of Noisy Quantum Computation,” no. arXiv:1810.03176. arXiv, Oct. 2018. doi: [10.48550/arXiv.1810.03176](https://doi.org/10.48550/arXiv.1810.03176).
- [13] E. Fontana, M. S. Rudolph, R. Duncan, I. Rungger, and C. Cîrstoiu, “Classical Simulations of Noisy Variational Quantum Circuits,” no. arXiv:2306.05400. arXiv, Jun. 2023. doi: [10.48550/arXiv.2306.05400](https://doi.org/10.48550/arXiv.2306.05400).
- [14] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.
- [15] D. Gottesman, *Stabilizer codes and quantum error correction*. California Institute of Technology, 1997.
- [16] A. R. Calderbank and P. W. Shor, “Good quantum error-correcting codes exist,” *Physical Review A*, vol. 54, no. 2, p. 1098, 1996.
- [17] F. Gaitan, *Quantum error correction and fault tolerant quantum computing*. CRC Press, 2008.
- [18] A. M. Steane, “Error correcting codes in quantum theory,” *Physical Review Letters*, vol. 77, no. 5, p. 793, 1996.
- [19] A. Steane, “Multiple-particle interference and quantum error correction,” *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 452, no. 1954, pp. 2551–2577, 1996.
- [20] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, “Topological quantum memory,” *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.

- [21] A. Y. Kitaev, “Fault-tolerant quantum computation by anyons,” *Annals of physics*, vol. 303, no. 1, pp. 2–30, 2003.
- [22] C. T. Chubb, “General tensor network decoding of 2D Pauli codes,” *arXiv preprint arXiv:2101.04125*, 2021.